製造指示アプリケーションモジュール — 機能設計

> **2026-10-24 改版（WI-017）**。本ファイルは `001_DD-FN_製造指示登録・編集.md` を置き換える。旧ファイルは承認済みの前版として変更しない。変更内容：製造指示の備考の上限を1000文字とする（REQ-090。REQ-016 の500文字を置き換える）。詳細は `work-items/WI-017/` を参照。

001_DD-FN — 001_DD と 001_DD-API（および後の画面B の一覧 DD）が使用する。要件 REQ-010〜REQ-018。

改版履歴

項目	値
文書ID	001_DD-FN
システム名	ProductionManagementAI
サブシステム名	製造指示
作業項目	WI-002
作成者	Claude (for ThanhTN)
作成日	2026-09-18
最終更新者	Claude (for ThongTM)
最終更新日	2026-10-24

版	日付	作成者	改版内容
1	2026-09-18	Claude (for ThanhTN)	新規作成。001_DD 版1 のモジュール3〜5（サービス、工場時計、指示番号採番）を本書へ移動
2	2026-09-18	Claude (for ThanhTN)	実装と整合（計画版2）：生成値は再読込ではなく RETURNING で取得。読込時の `xmin` を同時実行制御の元値とする。`AllowedNext()` は拡張メソッド
3	2026-09-22	Claude (for ThanhTN)	WI-004 REQ-033：UpdateAsync の手順7 でエンティティを通じて完了日時を記録。新しい手順や失敗経路はなし
4	2026-10-24	Claude (for ThongTM)	WI-017（REQ-090〜REQ-092）：備考の上限を500文字から1000文字へ変更。製造指示 DTO の `Notes` の最大長を 1000、`ProductionOrder.MaxNotesLength` = 1000。手順と失敗経路の変更なし。ファイル名に日付を付与（WI-017 DEC-002）

概要とメソッド一覧

項目	値
モジュール名	`ProductionOrderService` と補助サービス `PlantClock`、`OrderNumberIssuer`（`ProductionManagementAI.Application.ProductionOrders`。2つのインターフェースの実装は `ProductionManagementAI.Infrastructure.ProductionOrders`）
概要	製造指示のアプリケーション層のユースケース：入力をチェックし、ドメインルールを適用し、リポジトリで永続化し、API が HTTP レスポンスに変換する型付きの結果を返す。時計と採番は共通部品のため、画面B（一覧、納期遅れ表示）でも再利用する

No	メソッド名	概要	備考
1	`ProductionOrderService.ListProductsAsync`	選択用の製品を SKU 順で返す	API-PRD-01
2	`ProductionOrderService.GetAsync`	1件の製造指示を `ProductionOrderResponse` で返す	API-PO-02
3	`ProductionOrderService.CreateAsync`	チェック、採番、挿入	API-PO-01
4	`ProductionOrderService.UpdateAsync`	チェック、版の確認、ドメインルール、保存	API-PO-03
5	`ProductionOrderMapper.ToResponse`	エンティティ → レスポンス DTO（値の変換は後述）	内部、static
6	`IPlantClock.Today` / `CurrentYear`	工場ローカルの日付と年（`Asia/Tokyo`）	共通
7	`IOrderNumberIssuer.NextAsync`	採番 upsert による年ごとの連番	共通

すべてのサービスメソッドは `CancellationToken` を受け取り、`Result<T>` を返す。これは `Ok(T)`、`Invalid(errors)`、`NotFound`、`Conflict`、`RuleViolation(code)` の判別共用体である。コントローラーはそれぞれを 200/201、400、404、409、422 に変換する（DEC-023）。予期しない例外は `Result` にせず、全体の Problem Details ハンドラー（500、MSG-E013）へ伝播させる。

共通ユーティリティ参照

No	名前	概要	備考
1	`TimeProvider`（.NET）	注入する時刻ソース。テストでは `FakeTimeProvider`	`TimeProvider.System` として登録
2	`IProductionOrderRepository`	EF Core のデータアクセス：`ListProductsAsync`、`ProductExistsAsync`、`FindAsync(id, tracked)`、`Add`、`SaveChangesAsync`、`BeginTransactionAsync`	Infrastructure。列を射影する単一テーブルのクエリ（001_DD「データベースとトランザクションの対応」）
3	`ProductionOrderTelemetry`	`ProductionManagementAI.ProductionOrders` という名前の `ActivitySource` + `Meter`	001_DD「可観測性」
4	`ILogger<ProductionOrderService>`	`LoggerMessage` のソース生成メソッドによる構造化ログ	備考の本文はログに出さない

リクエストデータ

本モジュールが（リポジトリと採番を通じて）データベースへ送るパラメータ。すべてパラメータ化し、SQL に文字列連結しない（`ai/rules/database.md`）。

No	名前	変数名	型	長さ	使用するメソッド	備考
1	製造指示ID	`id`	`Guid`	—	GetAsync, UpdateAsync	主キー検索
2	製品ID	`productId`	`Guid`	—	CreateAsync, UpdateAsync	`ProductExistsAsync`
3	指示年	`year`	`short`	—	CreateAsync → NextAsync	採番 upsert の `@year`
4	元の行バージョン	`xmin`	`uint`	—	UpdateAsync	EF が `WHERE xmin = @original` を付加
5	エンティティの値	`ProductionOrder`	entity	—	CreateAsync, UpdateAsync	EF の変更追跡による INSERT / UPDATE

レスポンス／値の変換

`ProductionOrderResponse`（項目一覧は 001_DD-API）は `ProductionOrderMapper.ToResponse` で作成する：

No	名前	変数名	型	長さ	必須	値の変換	備考
1	ID	`Id`	`Guid`	—	yes	`entity.Id`	
2	指示番号	`OrderNumber`	`string`	13	yes	`entity.OrderNumber`（DB で生成。挿入時に EF が `RETURNING` で取得）	
3	製品ID	`ProductId`	`Guid`	—	yes	保存値のまま	
4	数量	`Quantity`	`int`	—	yes	保存値のまま	
5	納期	`DueDate`	`DateOnly`	—	yes	保存値のまま	`YYYY-MM-DD` で直列化
6	ステータス	`Status`	`string`	—	yes	`entity.Status.ToString()`	
7	遷移可能な次のステータス	`AllowedNextStatuses`	`string[]`	—	yes	`entity.Status.AllowedNext()`（`ProductionOrderStatusExtensions`）、enum の順	M-02
8	製品・数量を編集可能	`IsProductQuantityEditable`	`bool`	—	yes	`entity.Status == Draft`	REQ-018
9	備考	`Notes`	`string?`	1000	yes	保存値のまま（空は `null`）	
10	登録日時	`CreatedAt`	`DateTimeOffset`	—	yes	`created_at_utc`	UTC

No	名前	変数名	型	長さ	必須	値の変換	備考
11	更新日時	`UpdatedAt`	`DateTimeOffset`	—	yes	`updated_at_utc`	UTC
12	版	`Version`	`uint`	—	yes	`entity.RowVersion`（`xmin`）	DEC-014

`ProductResponse` = `{ Id, Sku, Name }`。クエリで直接射影する。

メソッド設計

1. ProductionOrderService.ListProductsAsync

項目	値
説明	製品の選択欄のため、全製品を SKU 順で返す
戻り値の型	`Task<IReadOnlyList<ProductResponse>>`
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	`CancellationToken`	`ct`	

戻り値

型	名前	説明
`IReadOnlyList<ProductResponse>`	products	現在の初期データで30行。空の一覧も有効（クライアントで MSG-E014）

処理概要：追跡なしの射影を1回実行する。ページングなし（DEC-019、DEC-022）。

処理フロー

手順	説明	呼び出し
1	`SELECT id, sku, name FROM products ORDER BY sku`	`IProductionOrderRepository.ListProductsAsync`
2	一覧を返す	—

2. ProductionOrderService.GetAsync

項目	値
説明	編集画面のため1件の製造指示を読み込む
戻り値の型	`Task<Result<ProductionOrderResponse>>`
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	`Guid`	`id`	ルートの製造指示ID
2	`CancellationToken`	`ct`	

戻り値

型	名前	説明
`Result<ProductionOrderResponse>`	result	`Ok` または `NotFound`

処理概要：`xmin` を含めて追跡なしで読み込み、変換する。

処理フロー

手順	説明	呼び出し
1	`FindAsync(id, tracked: false)`	リポジトリ
2	見つからない → `NotFound`	—
3	`Ok(ToResponse(order))`	`ProductionOrderMapper.ToResponse`

3. ProductionOrderService.CreateAsync

項目	値
説明	次の `PO-YYYY-NNNNN` 番号で `Draft` の製造指示を登録する（REQ-010）
戻り値の型	`Task<Result<ProductionOrderResponse>>`
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	`CreateProductionOrderRequest`	`request`	`ProductId?`、`Quantity?`、`DueDate?`、`Notes?`（項目の欠落をバインドエラーではなく「必須」として報告するため nullable）
2	`CancellationToken`	`ct`	

戻り値

型	名前	説明
`Result<ProductionOrderResponse>`	result	`Ok`（コントローラーは 201 ＋ `Location` を返す）または `Invalid(errors)`

処理概要：データベースに触れる前にすべての項目エラーを集める。次にデータ依存のチェックを行う。その後1つのトランザクションで採番と挿入を行うため、挿入に失敗しても番号を消費しない（001_DB）。

処理フロー

手順	説明	呼び出し
1	スパン `ProductionOrder.Create` を開始	`ProductionOrderTelemetry`
2	必須チェック：`ProductId`（MSG-E001）、`Quantity`（MSG-E003）、`DueDate`（MSG-E004）	—
3	値のチェック：`ValidateQuantity`（MSG-E003 / MSG-E010）、`NormalizeNotes`（前後空白を除去、空 → null）＋ `ValidateNotes`（MSG-E006）	`ProductionOrder` の static チェック関数（001_DD モジュール1）
4	データのチェック（エラーのない項目のみ）：`ProductExistsAsync`（MSG-E002）、`DueDate < IPlantClock.Today` → MSG-E005	リポジトリ、`IPlantClock.Today`

手順	説明	呼び出し
5	エラーあり → `Invalid(errors)`。カウンター `created{outcome=validation_failed}`、ログ `ProductionOrderValidationFailed`（項目名のみ）	—
6	`BeginTransactionAsync`	リポジトリ
7	`seq = NextAsync(IPlantClock.CurrentYear)`	`IOrderNumberIssuer.NextAsync`
8	`order = ProductionOrder.Create(productId, quantity, dueDate, notes, year, seq, utcNow)`	ドメイン
9	`Add(order)`、`SaveChangesAsync`、commit	リポジトリ
10	生成された `order_number` と新しい `xmin` は既にエンティティにある：`SaveChangesAsync` 中に EF Core が `RETURNING` で取得する（追加のクエリなし）	EF Core
11	カウンター `created{outcome=success}`、スパン属性 `production_order.id`、`production_order.number`、`Ok(ToResponse(order))`	テレメトリー、マッパー

手順7〜9 での失敗（例：年間100,000件目での採番の CHECK 違反）→ ロールバック、カウンター `created{outcome=error}`、Error ログ、再スロー → 500 MSG-E013（DEC-013）。

4. ProductionOrderService.UpdateAsync

項目	値
説明	古い内容での保存を防ぎつつ、編集とステータス変更を保存する（REQ-011、REQ-017、REQ-018、DEC-010）
戻り値の型	`Task<Result<ProductionOrderResponse>>`
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	`Guid`	`id`	ルートの ID
2	`UpdateProductionOrderRequest`	`request`	`ProductId?`、`Quantity?`、`DueDate?`、`Status?`、`Notes?`、`Version?`
3	`CancellationToken`	`ct`	

戻り値

型	名前	説明
`Result<ProductionOrderResponse>`	result	`Ok`、`Invalid`、`NotFound`、`Conflict`、`RuleViolation("MSG-E007" | "MSG-E008")`

処理概要：チェックの順序を固定し、各失敗の結果を1つに定める：形式（400）→ 存在（404）→ 版（409）→ ドメインルール（422）→ データ依存のチェック（400）→ 保存時の競合（409）。すべてのチェックを通過しない限り何も書き込まない（`SaveChangesAsync` は1回、DEC-007）。

処理フロー

手順	説明	呼び出し
1	スパン `ProductionOrder.Update` を開始	テレメトリー
2	登録の手順2〜3 と同じ必須・値のチェック。加えて `Status` は必須で既知の enum 値（不明なら MSG-E007）、`Version` は必須（MSG-E009）	ドメインのチェック関数
3	エラー → `Invalid`（`updated{outcome=validation_failed}`）	—
4	`FindAsync(id, tracked: true)`。なし → `NotFound`（`updated{outcome=not_found}`）	リポジトリ
5	`request.Version != order.RowVersion` → `Conflict`（`updated{outcome=conflict}`、ログ `ProductionOrderConcurrencyConflict`）	—
6	`fromStatus`、`dueDateChanged = request.DueDate != order.DueDate`、`productChanged = request.ProductId != order.ProductId` を記憶	—
7	`order.Update(productId, quantity, dueDate, notes, status, utcNow)`。`DomainRuleViolation(code)` を捕捉 → `RuleViolation(code)`（`updated{outcome=rule_violation}`、ログ `ProductionOrderRuleViolated`）。例外時エンティティは変更されない。`InProgress → Completed` ではエンティティが `CompletedAtUtc = utcNow` も設定する（001_DD モジュール1 手順5、WI-004 REQ-033）。後の手順が失敗すれば何も保存されないため、完了日時も記録されない	ドメイン（001_DD モジュール1）
8	`productChanged` なら `ProductExistsAsync`（MSG-E002）。`dueDateChanged` なら `DueDate < IPlantClock.Today` → MSG-E005（DEC-009）。エラー → `Invalid`。追跡中の変更は破棄される（コンテキストはリクエスト単位で保存しない）	リポジトリ、`IPlantClock`
9	`SaveChangesAsync`。EF が `WHERE xmin = @original` を付加する。元値は手順4 で読み込んだ `xmin`（手順5 で `request.Version` と一致を確認済み）なので、手順4〜9 の間の書き込みも検出できる	リポジトリ
10	`DbUpdateConcurrencyException` → `Conflict`	—
11	`status != fromStatus` なら：カウンター `status_transitions{from,to}`、スパン `status.from` / `status.to`	テレメトリー
12	新しい `xmin` は `RETURNING` で取得 → `Ok(ToResponse(order))`、`updated{outcome=success}`	マッパー

5. ProductionOrderMapper.ToResponse

項目	値
説明	エンティティから `ProductionOrderResponse` への純粋な変換
戻り値の型	`ProductionOrderResponse`
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	`ProductionOrder`	`order`	読み込んだエンティティ

戻り値

型	名前	説明
`ProductionOrderResponse`	response	「レスポンス／値の変換」を参照

処理概要：項目ごとのコピーと2つの計算項目。I/O なし。

処理フロー

手順	説明	呼び出し
1	保存項目をコピー	—
2	`AllowedNextStatuses` と `IsProductQuantityEditable` を計算	`ProductionOrderStatusExtensions.AllowedNext`

6. IPlantClock.Today / CurrentYear

項目	値
説明	`Plant:TimeZone` で設定したタイムゾーン（既定 `Asia/Tokyo`、DEC-011、DEC-017）での工場ローカルの日付と年
戻り値の型	`DateOnly` / `short`
作成者／日付	Claude / 2026-09-18

項目	値
最終更新者／日付	—

引数：なし。コンストラクターの依存：`TimeProvider`、`IOptions<PlantOptions>`。

戻り値

型	名前	説明
`DateOnly`	`Today`	`DateOnly.FromDateTime(TimeZoneInfo.ConvertTime(timeProvider.GetUtcNow(), zone).DateTime)`
`short`	`CurrentYear`	`(short)Today.Year`

処理概要：`TimeZoneInfo.FindSystemTimeZoneById(options.TimeZone)` は生成時に1回だけ解決する。.NET の ICU サポートにより IANA の ID は Linux と Windows の両方で使える。起動時のオプション検証で不明な ID を拒否するため、リクエストごとではなく起動時に失敗する。

処理フロー

手順	説明	呼び出し
1	現在の UTC 時刻を読む	`TimeProvider.GetUtcNow`
2	工場のタイムゾーンに変換し、日付／年を取る	`TimeZoneInfo.ConvertTime`

7. IOrderNumberIssuer.NextAsync

項目	値
説明	呼び出し元のトランザクション内で、001_DB の採番 upsert を使って年の次の `order_seq` を返す（DEC-013）
戻り値の型	`Task<int>`
作成者／日付	Claude / 2026-09-18
最終更新者／日付	—

引数

No	型	名前	説明
1	`short`	`year`	工場の年

No	型	名前	説明
2	`CancellationToken`	`ct`	

戻り値

型	名前	説明
`int`	`seq`	1〜99999

処理概要：`Database.SqlQuery<int>($"INSERT ... ON CONFLICT (order_year) DO UPDATE SET last_seq = ... + 1 RETURNING last_seq")`。EF は補間した SQL をパラメータに変換する。行ロックは呼び出し元がコミットするまで保持され、同じ年の登録を直列化する。

処理フロー

手順	説明	呼び出し
1	`Database.CurrentTransaction` が null なら `InvalidOperationException` をスロー（挿入のトランザクション外での採番を防ぐ）	—
2	upsert を実行し、`last_seq` を返す	001_DB `production_order_number_counters`
3	CHECK 違反（`last_seq > 99999`）は `PostgresException` 23514 になる → 呼び出し元がロールバック → 500	—

未決事項

なし。関連する決定：`work-items/WI-002/decisions.md` の DEC-009、DEC-010、DEC-013、DEC-014、DEC-017、DEC-023。